

UNIVERSIDAD SALESIANA DE BOLIVIA



MANUAL DE INSTALACIÓN

ControlFuel

Sistema embebido con ESP32 para el control de acceso, medición de nivel de gasolina y racionamiento inteligente de combustible en tanques subterráneos de estaciones de servicio.

INTEGRANTES:

- Oliver Choque Pachacuti
- Danner Suxo Quispe
- Franklin Adhemar Pongo Cori
- Willka Daniel Condori Ventura
- Andrei Gabriel Rada Brun
- Alan Jhohan Del Rio Lopez

Materia: Ingeniería de Sistemas I

Semestre: 8vo Semestre

Paralelo: 8 - 111

LA PAZ – BOLIVIA

2026

ÍNDICE GENERAL

1. Introducción y Requisitos Previos	3
1.1. Propósito del documento.....	3
1.2. Requisitos de hardware.....	3
1.3. Requisitos de software.....	3
2. Ensamblaje Físico del Hardware (Circuito)	4
2.1. Conexión del ESP32 y el Lector RFID (RC522).....	4
2.2. Conexión del sensor ultrasónico (HC-SR04 / JSN-SR04T).....	4
2.3. Conexión del módulo relé y actuador de prueba.....	4
3. Despliegue del Backend y Base de Datos	4
3.1. Configuración del servidor local (XAMPP / Apache / MySQL).....	4
3.2. Creación e importación del esquema SQL (schema.sql).....	4
3.3. Resolución de dependencias y configuración de la base de datos....	5
3.4. Configuración de credenciales y API Keys.....	5
4. Configuración y Carga del Firmware	5
4.1. Preparación del entorno (Arduino IDE / PlatformIO) y librerías.....	5
4.2. Configuración de parámetros de red y servidor en el código.....	5
4.3. Compilación y carga en el ESP32.....	5
5. Solución de Problemas (Troubleshooting)	6
5.1. Fallas de red y conexión Wi-Fi / API.....	6
5.2. Errores de lectura en el lector RFID.....	6
5.3. Inestabilidad en el sensor ultrasónico.....	6

MANUAL DE INSTALACIÓN Y DESPLIEGUE — CONTROLFUEL

Proyecto Integrador: Sistema embebido con ESP32 para el control de acceso, medición de nivel de gasolina y racionamiento inteligente de combustible en tanques subterráneos de estaciones de servicio.

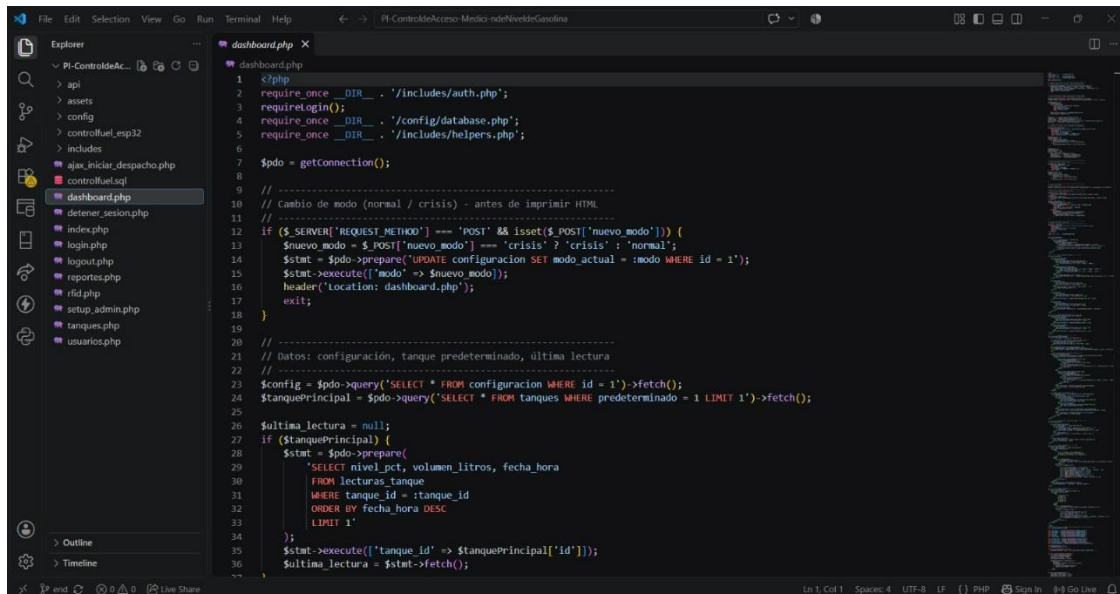
CONTENIDO DESARROLLADO: MANUAL DE INSTALACIÓN

1. Introducción y Requisitos Previos

- **1.1. Propósito del documento:** Guiar paso a paso al personal técnico en el montaje físico, configuración del entorno de servidor, carga de base de datos y programación del microcontrolador ESP32 para el funcionamiento de ControlFuel en un entorno de laboratorio.
- **1.2. Requisitos de hardware:**
 - Microcontrolador ESP32 DevKit v1.
 - Lector RFID RC522 con tarjetas/llaveros MIFARE.
 - Sensor ultrasónico HC-SR04 o JSN-SR04T.
 - Módulo relé de 1 canal y fuente de alimentación regulada (3.3V/5V).
 - Computadora de prueba con puerto USB y conexión Wi-Fi.
- **1.3. Requisitos de software:**
 - Servidor web Apache y MySQL (vía XAMPP o nativo) con PHP 8.x (Extensiones pdo y pdo_mysql habilitadas).
 - Entorno de desarrollo Arduino IDE o PlatformIO.
 - Librerías de firmware: WiFi, HTTPClient, ArduinoJson, SPI, MFRC522.

2. Ensamblaje Físico del Hardware

- **2.1. Conexión ESP32 y RFID:** Conectar los pines del RC522 por interfaz SPI respetando los voltajes lógicos: SDA/SS, SCK, MOSI, MISO, RST, GND y **3.3V estrictos** (evitar 5V para no dañar el circuito).
- **2.2. Conexión Sensor Ultrasónico:** Conectar los pines Trig y Echo a los pines digitales correspondientes configurados en el código del ESP32, asegurando una correcta alineación cenital en el tanque de prueba.



```
1 <?php
2 require_once __DIR__ . '/includes/auth.php';
3 requireLogin();
4 require_once __DIR__ . '/config/database.php';
5 require_once __DIR__ . '/includes/helpers.php';
6
7 $pdo = getConnection();
8
9 // Cambio de modo (normal / crisis) - antes de imprimir HTML
10 //
11 if ($SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['nuevo_modo'])) {
12     $nuevo_modo = $_POST['nuevo_modo'] === 'crisis' ? 'crisis' : 'normal';
13     $stmt = $pdo->prepare('UPDATE configuracion SET modo_actual = :modo WHERE id = 1');
14     $stmt->execute(['modo' => $nuevo_modo]);
15     header('location: dashboard.php');
16     exit;
17 }
18
19 //
20 // Datos: configuración, tanque predeterminado, última lectura
21 //
22 $config = $pdo->query('SELECT * FROM configuracion WHERE id = 1')->fetch();
23 $tanquePrincipal = $pdo->query('SELECT * FROM tanques WHERE predeterminado = 1 LIMIT 1')->fetch();
24
25 $ultima_lectura = null;
26 if ($tanquePrincipal) {
27     $stmt = $pdo->prepare(
28         'SELECT nivel_pct, volumen_litros, fecha_hora
29          FROM lecturas_tanque
30          WHERE tanque_id = :tanque_id
31          ORDER BY fecha_hora DESC
32          LIMIT 1'
33     );
34     $stmt->execute(['tanque_id' => $tanquePrincipal['id']]);
35     $ultima_lectura = $stmt->fetch();
36 }
```

- **2.3. Conexión del Módulo Relé:** Conectar la señal de control a un GPIO seguro del ESP32 y los contactos de potencia al actuador de prueba. El sistema debe garantizar un estado seguro (apagado) por defecto ante fallas de energía.

3. Despliegue del Backend y Base de Datos

- **3.1. Configuración del servidor local (XAMPP / Apache / MySQL):** Copiar la carpeta controlfue/ dentro del directorio htdocs/ de XAMPP y activar los servicios de Apache y MySQL.
- **3.2. Creación e importación del esquema SQL (schema.sql):** Crear la base de datos e importar el esquema ejecutando:

```
CREATE DATABASE controlfue CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

Y cargar el archivo database/schema.sql.



Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
configuracion	Examinar Estructura Buscar Insertar Vaciar Eliminar	1	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
despachos	Examinar Estructura Buscar Insertar Vaciar Eliminar	129	InnoDB	utf8mb4_unicode_ci	80.0 KB	-
lecturas_tanque	Examinar Estructura Buscar Insertar Vaciar Eliminar	529	InnoDB	utf8mb4_unicode_ci	80.0 KB	-
lectura_nivel	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
modo_operacion	Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
rol	Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
sesiones_despacho	Examinar Estructura Buscar Insertar Vaciar Eliminar	21	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
tanque	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
tanques	Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
tarjetas_rfid	Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
tarjeta_rfid	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
usuario	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	64.0 KB	-
usuarios	Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
13 tablas	Número de filas	690	InnoDB	utf8mb4_unicode_ci	544.0 KB	0 B

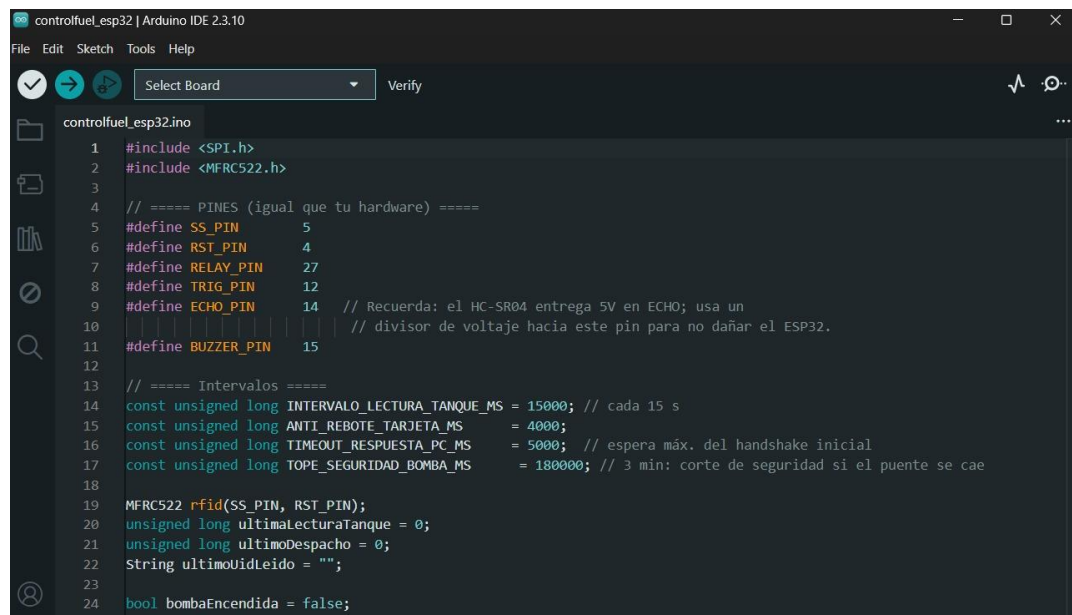
- **3.3. Resolución de dependencias y configuración de la base de datos:** Agregar la columna faltante en la tabla de configuración para evitar errores de cálculo en el modo normal:

```
ALTER TABLE configuracion ADD COLUMN
litros_normal_max DECIMAL(6,2) NOT NULL DEFAULT
10.00 AFTER limite_crisis_litros;
```

- **3.4. Configuración de credenciales y API Keys:** Editar el archivo config/database.php con los accesos locales y definir una API Key en config/api.php. Acceder una sola vez a http://localhost/controlfue/Setup_admin.php para generar el usuario administrador inicial, y luego restringir dicho archivo.

4. Configuración y Carga del Firmware

- **4.1. Preparación del entorno (Arduino IDE / PlatformIO) y librerías:** Abrir el proyecto en Arduino IDE con las librerías previamente instaladas.
- **4.2. Configuración de parámetros de red y servidor en el código:** Modificar en el código fuente las variables de red (ssid, password), la URL del backend (serverUrl) apuntando a la IP local del servidor y la misma API Key registrada en el backend.
- **4.3. Compilación y carga en el ESP32:** Conectar el ESP32 por USB, seleccionar la placa *ESP32 Dev Module*, compilar y cargar el código. Abrir el Monitor Serial a 115200 baudios para validar la conexión y el enlace HTTP.



```
controlfuelesp32.ino
1  #include <SPI.h>
2  #include <MFRC522.h>
3
4  // ===== PINES (igual que tu hardware) =====
5  #define SS_PIN      5
6  #define RST_PIN     4
7  #define RELAY_PIN   27
8  #define TRIG_PIN    12
9  #define ECHO_PIN    14 // Recuerda: el HC-SR04 entrega 5V en ECHO; usa un
10 // divisor de voltaje hacia este pin para no dañar el ESP32.
11 #define BUZZER_PIN  15
12
13 // ===== Intervalos =====
14 const unsigned long INTERVALO_LECTURA_TANQUE_MS = 15000; // cada 15 s
15 const unsigned long ANTI_REBOTE_TARJETA_MS      = 4000;
16 const unsigned long TIMEOUT_RESPUESTA_PC_MS    = 5000; // espera máx. del handshake inicial
17 const unsigned long TOPE_SEGURIDAD_BOMBA_MS    = 180000; // 3 min: corte de seguridad si el puente se cae
18
19 MFRC522 rfid(SS_PIN, RST_PIN);
20 unsigned long ultimaLecturaTanque = 0;
21 unsigned long ultimoDespacho = 0;
22 String ultimoUidLeido = "";
23
24 bool bombaEncendida = false;
```

5. Solución de Problemas (Troubleshooting)

- **Fallas de red y conexión Wi-Fi / API:** Si el ESP32 no conecta, verificar que la red Wi-Fi sea de 2.4 GHz y que las credenciales sean correctas. Un error 401 Unauthorized indica discrepancia en la API Key.
- **Errores de lectura en el lector RFID:** Si no lee tarjetas, revisar el cableado SPI y confirmar que el pin de alimentación esté en 3.3V.
- **Inestabilidad en el sensor ultrasónico:** Valores en 0 cm o inestables indican obstrucciones en el cono de propagación o inversión de pines Trig/Echo.